

COMPSCI 1DM3 - Winter 2026

Discrete Mathematics for Computer Science

Angela Zavaleta Bernuy



Last lecture

- A matrix is a rectangular array of numbers.
 - ▶ A matrix with m rows and n columns is called an $m \times n$ matrix.
 - ▶ Two matrices are equal if they have the same number of rows and the same number of columns and the corresponding entries in every position are equal.
- Addition of matrices is entry-wise, multiplication is using vector (inner) products
- $A^n = AA \cdots A$ (n times), A^{-1} denotes the inverse (if it exists), and A^t denotes the transpose.
- A matrix A is symmetric if $A^t = A$

Chapter 3

Algorithms

Algorithms

In this chapter, we use what we have learned so far to get computers to solve real-life problems.

- An **Algorithm** is a finite sequence of precise instructions for solving a problem or performing a computation.
- **Pseudocode** is a high-level description of the steps of an algorithm using informal (and self-explanatory) notations.
- Algorithms in scientific works, such as books and papers, are usually written in pseudocode.

Algorithm: find max

- Write an algorithm to find the largest element of a given sequence of n integers $\{a_1, a_2, \dots, a_n\}$.

```
1 function find_max(L, n):  
2     max_value = L[1]  
3     for i = 2 to n do  
4         if L[i] > max_value then  
5             max_value = L[i]  
6 return max_value
```

- Evaluate the above algorithm on the set $\{-1, 0, 5, 15, -3, 10\}$. $n=6$

$\text{max_value} = \cancel{-1} \cancel{0} \cancel{5} \textcircled{15}$

for $i = 2$ to 6

$i=2$ ① $a_2 = 0 > -1 \checkmark$

$i=3$ ② $a_3 = 5 > 0 \checkmark$

$i=4$ ③ $a_4 = 15 > 5 \checkmark$

$i=5$ ④ $a_5 = -3 > 15 \times$

$i=6$ ⑤ $a_6 = 10 > 15 \times$

Algorithm: Linear search

- Write an algorithm to locate a given element in a list L of n distinct elements.

```
1 function LinearSearch(L, n, target):  
2   for i = 1 to n do  
3     if L[i] = target then  
4       return i  
5   return -1 # Target not found
```

$L = [3, 12, 36]$, $n = 3$, $t = 12$

① $i = 1$, if $L[1] = 3 \neq t$

② $i = 2$, if $L[2] = 12 = t$ ✓
return ②

- How about when the input list is ~~ordered~~? For example:

$L = (1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22)$.

target = 14 $n = 16$

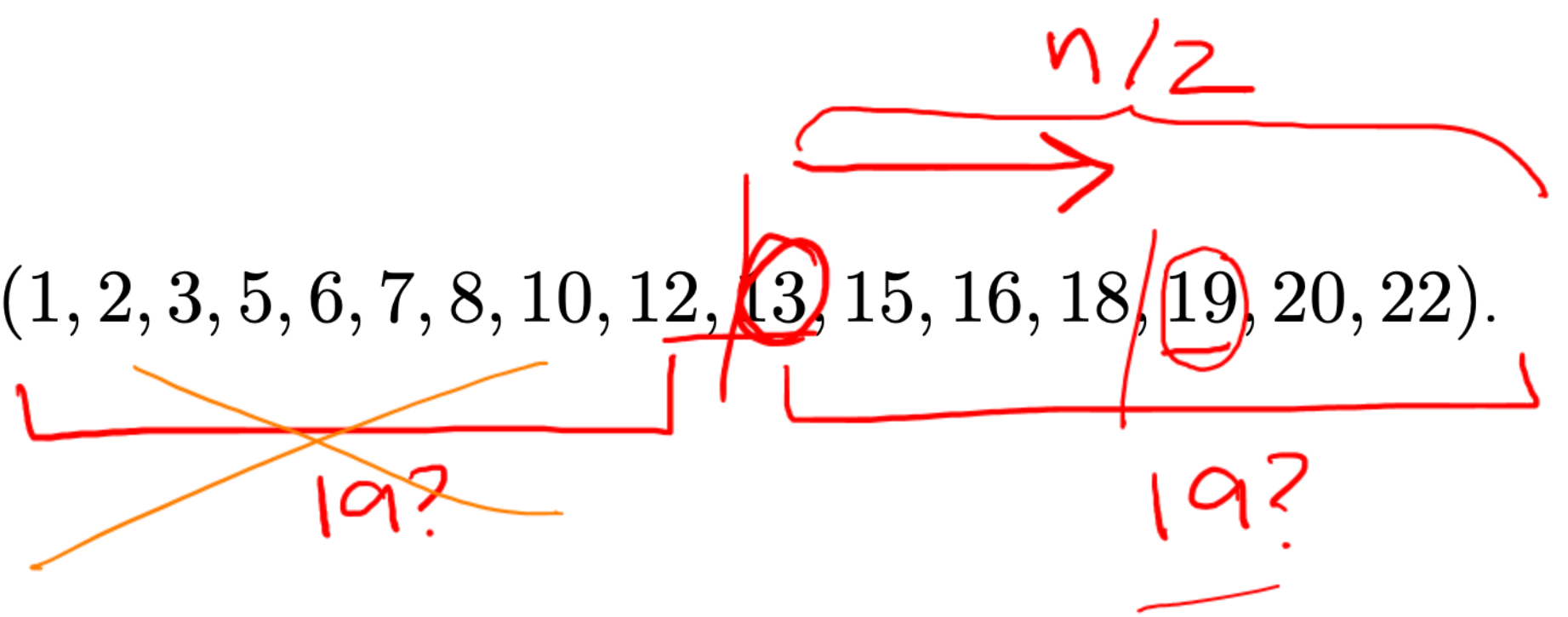
target = 4

$\neq$ target $\neq$ target

Algorithm: Binary search

Write an algorithm to locate 19 in (1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22).

ordered list.



- equal → return position
- greater → seek 2nd half
- less → seek 1st half.

Algorithm: Binary search

Write an algorithm to locate a given element in an ordered list L of n distinct elements.

```
1 function BinarySearch(L, n, target):
2     left = 1
3     right = n
4     while left <= right do
5         mid = floor((left + right) / 2)
6         if L[mid] = target then
7             return mid
8         if L[mid] < target then (seek 2nd half)
9             left = mid + 1      right = n
10        else L[mid] > target (seek 1st half)
11            right = mid - 1     left = 1
12    return -1 # Target not found
```

Algorithm: Binary search

Trace the Binary search algorithm for the element $x = 31$ in the list
(3, 5, 9, 13, 15, 17, 18, 31).

① $L[4] = 13 < \text{target}$

② $L[6] = 17 < \text{target}$

③ $L[7] = 18 < \text{target}$

④ $L[8] = 31 = \text{target}$

return 8

```
1 function BinarySearch(L, n, target):  
2     left = 1 = 4 + 1 = 5 = 6 + 1 = 7 = 7 + 1 = 8  
3     right = n = 8  
4     while left <= right do  
5         mid = floor((left + right) / 2) = 4 6 7 8  
6         if L[mid] = target then  
7             return mid  
8         if L[mid] < target then  
9             left = mid + 1  
10        else  
11            right = mid - 1  
12    return -1 # Target not found
```

Algorithm: Bubble sort

```
1 function BubbleSort(L, n):  
2   for i = 1 to n - 1 do  
3     for j = 1 to n - i do  
4       if L[j] > L[j + 1] then  
5         swap(L[j], L[j + 1])
```

when to swap, at the end of the list is sorted.
pass

i = 1

j = 1 → 4

Start	3	2	4	1	5
Pass 1	3	2	4	1	5
	2	3	4	1	5
	2	3	4	1	5
	2	3	1	4	5
Pass 2	2	3	1	4	5
	2	3	1	4	5
	2	1	3	4	5
Pass 3	2	1	3	4	5
	1	2	3	4	5

Algorithm: Insertion sort

```
1 function InsertionSort(L, n):  
2   for i = 2 to n do  
3     key = L[i]  
4     j = i - 1  
5     while j >= 1 and L[j] > key do  
6       L[j + 1] = L[j]  
7       j = j - 1  
8     L[j + 1] = key
```

→ where to insert the key.

→ iterate through unsorted elements.

Start 3 2 4 1 5

Insert 2 3 2 4 1 5

Insert 4 2 3 4 1 5

Insert 1 2 3 4 1 5

Insert 5 1 2 3 4 5

Sorted 1 2 3 4 5

???